Module 5

Deep Learning

(Deep Neural Networks)

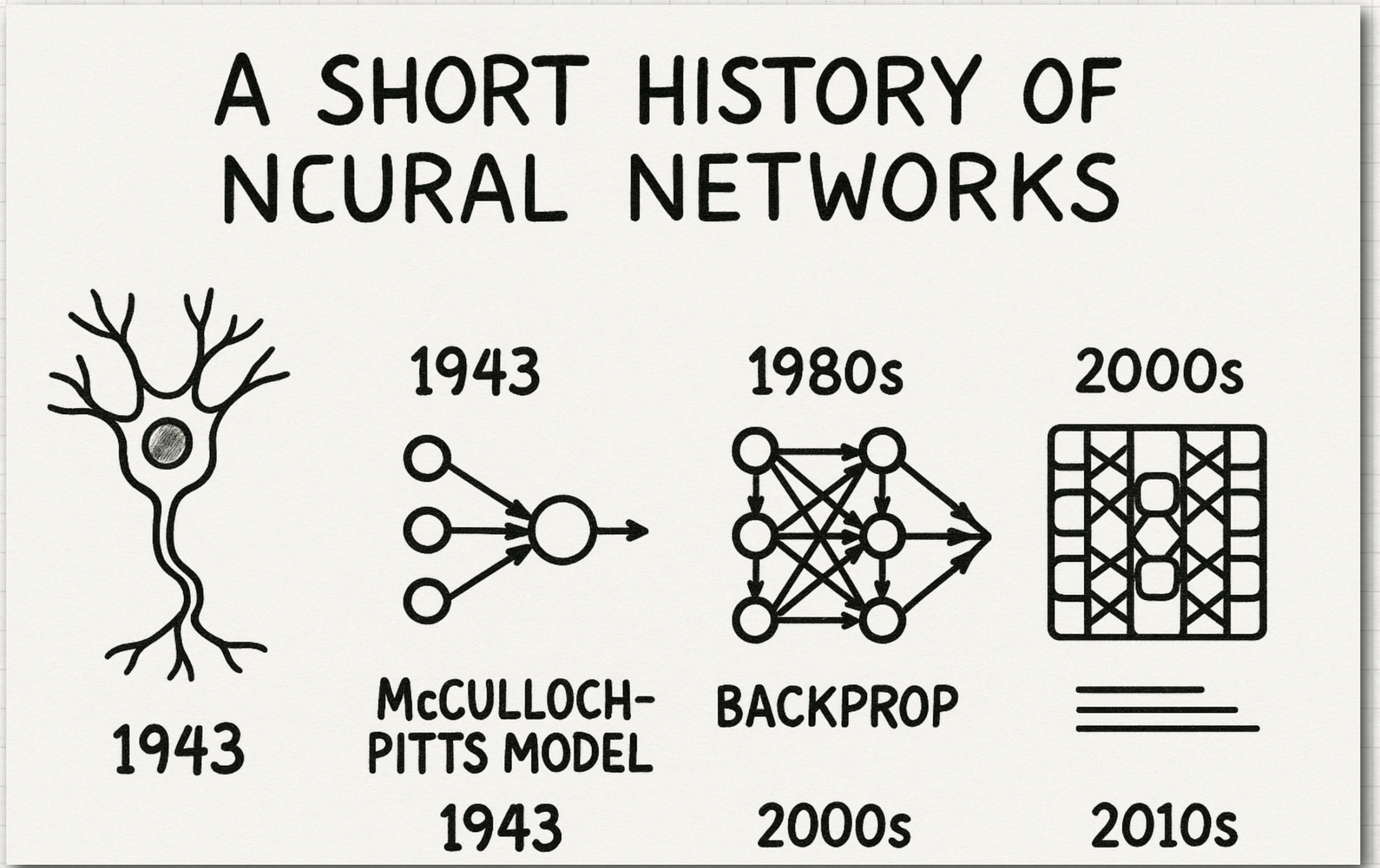
Resources

PyTorch a 60-minute blitz

Goodfellow et al. (2016) Deep Learning

Deep Neural Networks

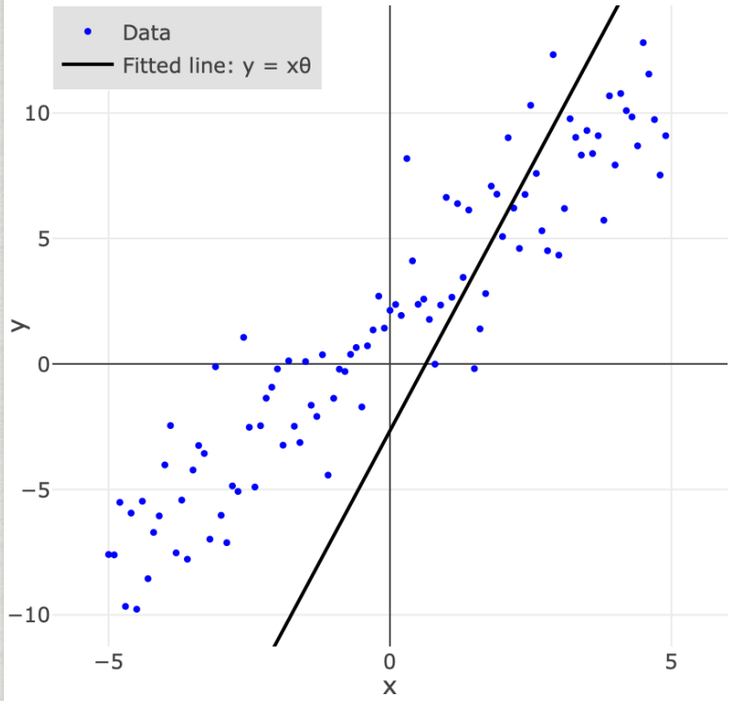
A very short history



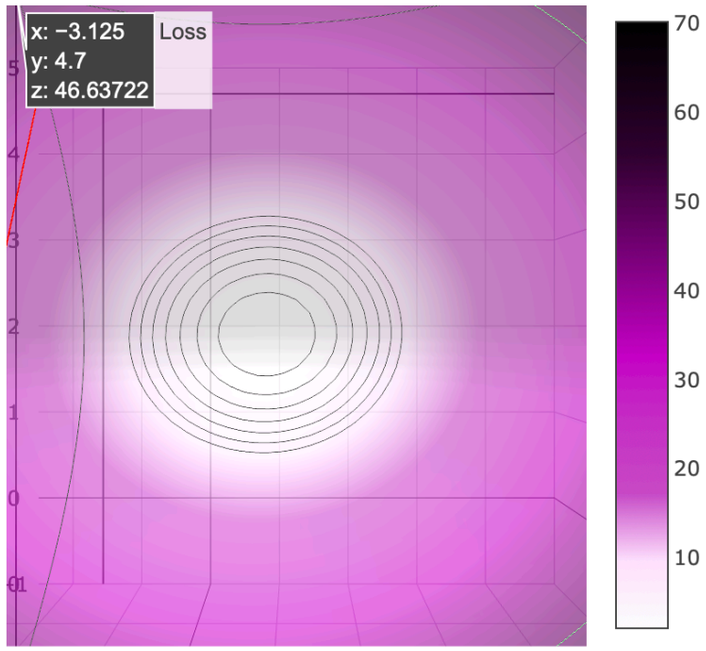
Learning through back-propagation

Gradient Descent

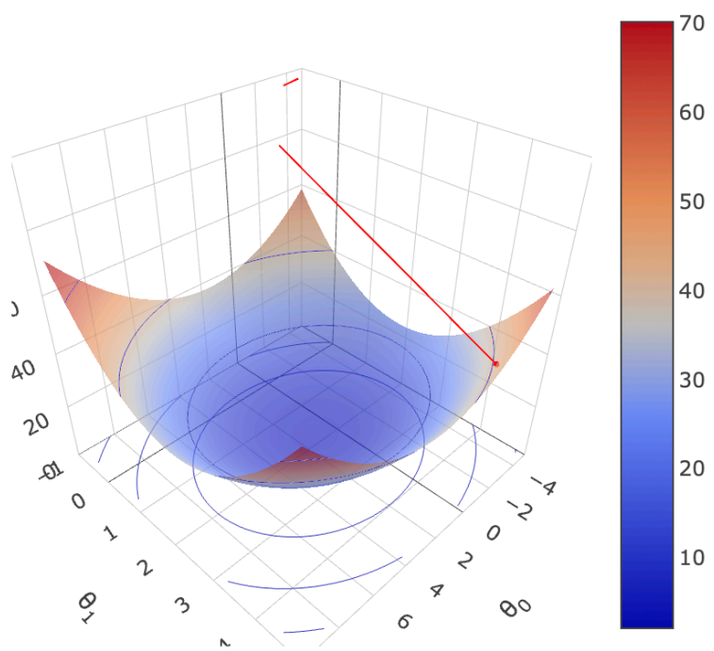
Labelled Data & Model Output



Contour Loss Function and Descent Path



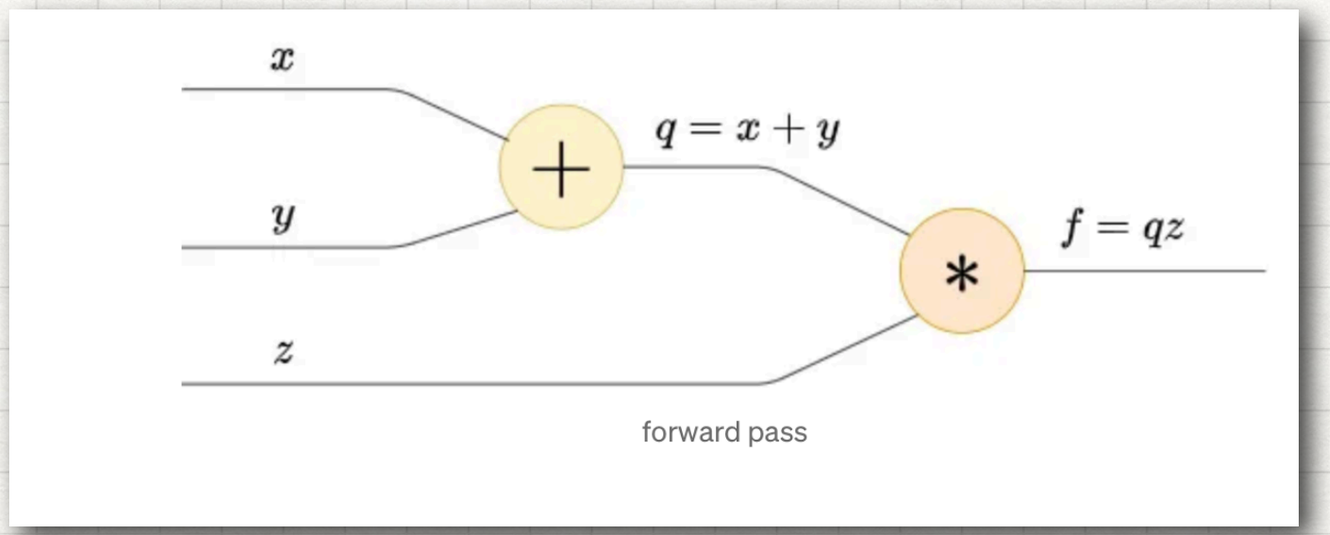
3D Loss Function and Descent Path

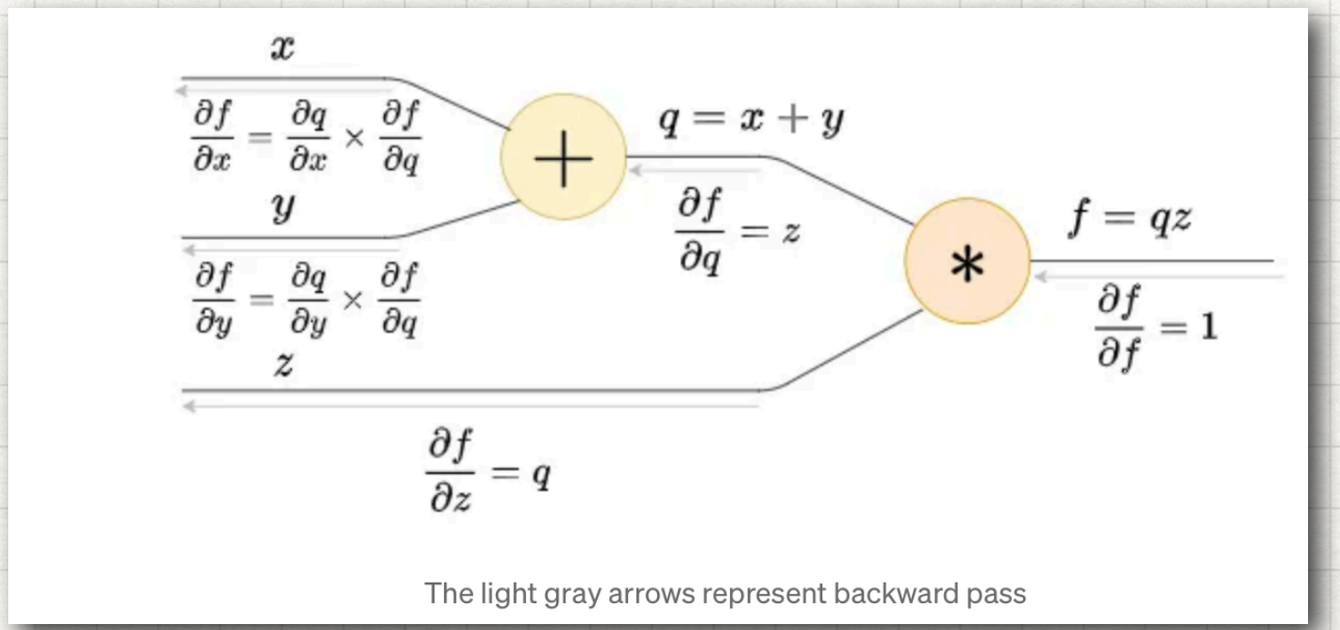


https://aero-learn.imperial.ac.uk/vis/Machine Learning/gradient_descent_3d.html

Learning through back-propagation

Credit assignment & backprop





Support for Deep Learning in Python

PyTorch:

$ conda activate cog-ai

$ pip install torch torchvision

Tensors (Like numpy arrays, but better):

x = torch.tensor([2.0, 3.0])

Autograd:

x = torch.tensor([2.0, 3.0], requires_grad=True)

torch.nn:

A library containing different types of NN layers and operations on those layers:

e.g. nn.Linear, nn.Sequential, nn.ReLU, nn.Conv2D, etc.

DataLoader:

An iterable utility that wraps a Dataset to provide an easy and efficient way to access data in batches. It handles tasks like automatically grouping samples into batches, shuffling the data each epoch, and parallelizing the data loading process across multiple worker processes

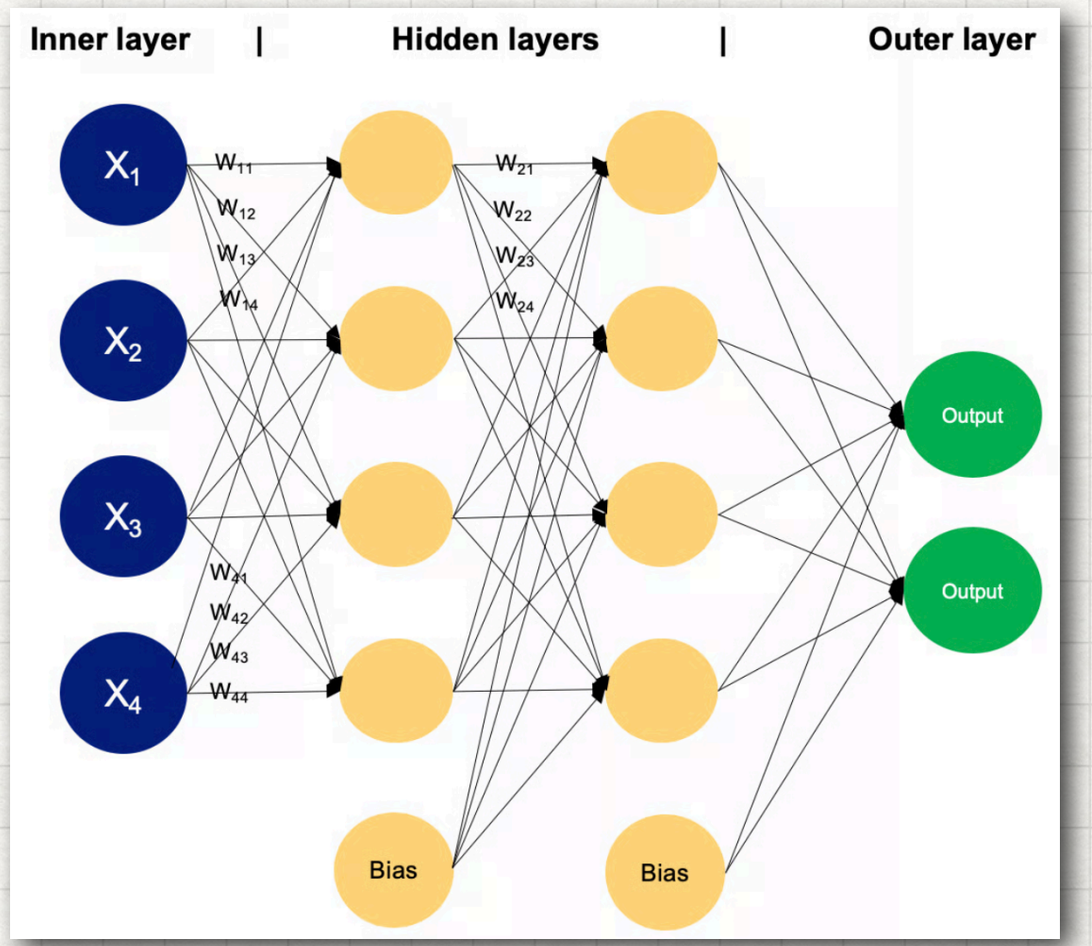
Mini-project 4: A small Neural Network

Train an MLP on MNIST and then analyse confusion patterns as a proxy for visual similarity.

MNIST Database

Mini-project 4: A small Neural Network

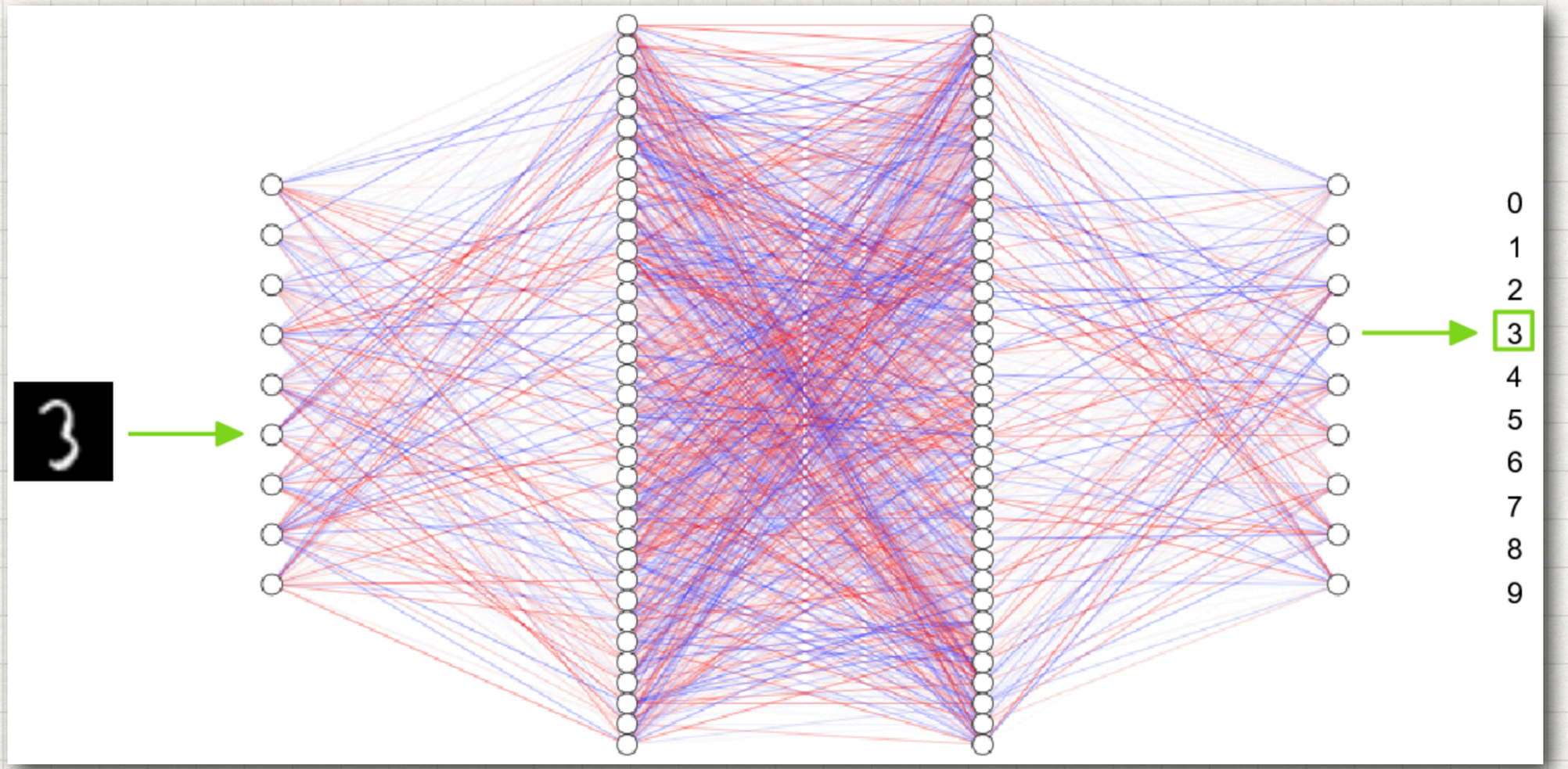
Train an MLP on MNIST and then analyse confusion patterns as a proxy for visual similarity.



Multi-layer Perceptron

Mini-project 4: A small Neural Network

Train an MLP on MNIST and then analyse confusion patterns as a proxy for visual similarity.



Demo 1: Using Tensors & Autograd

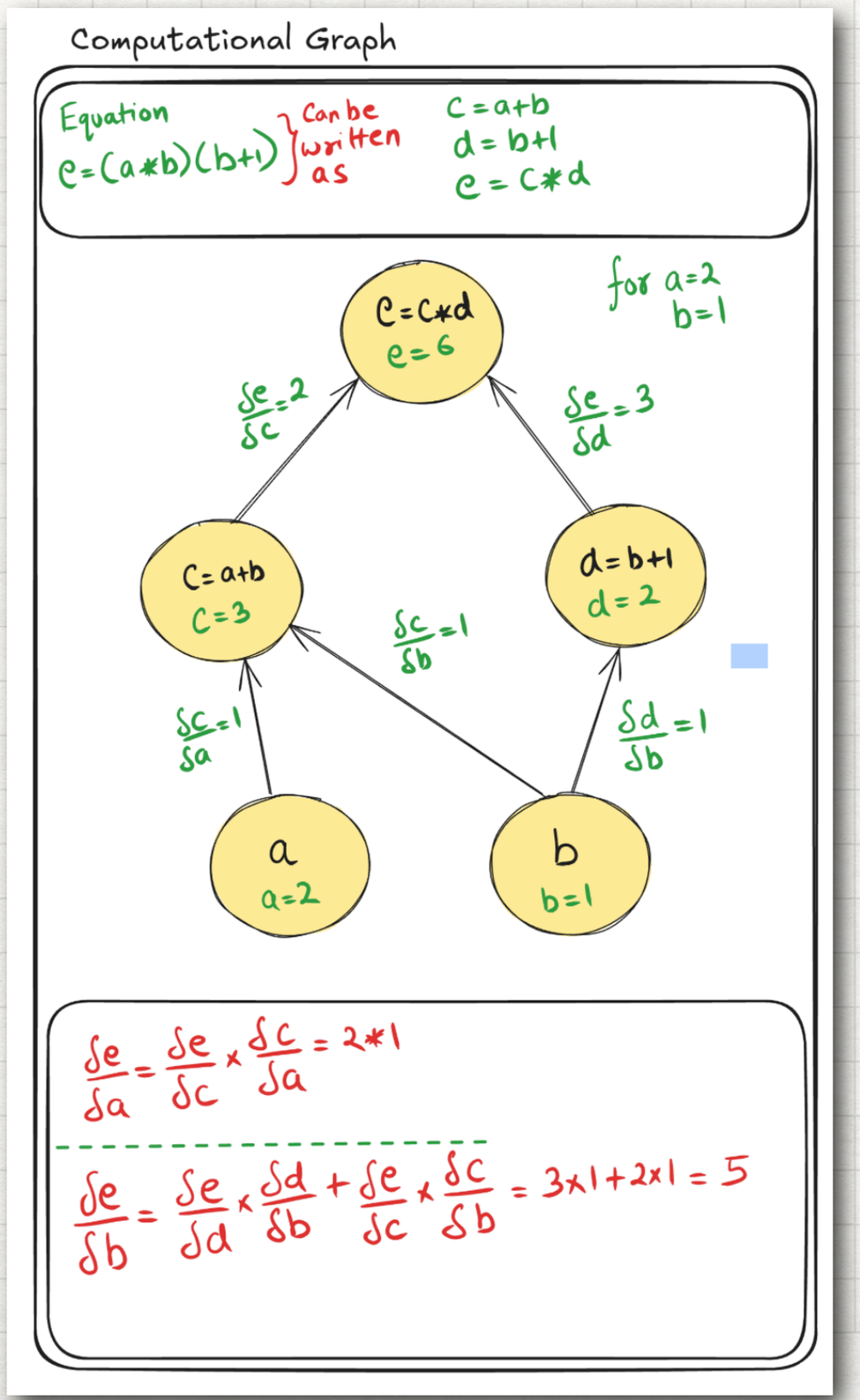
Define tensors ‘a’ and ‘b’. Perform series of calculations on these tensors, using variables ‘c’, ‘d’, and ‘e’ to store outputs. Then print the gradient of ‘e’ with respect to ‘a’ and ‘b’.

$$c = a + b$$
$$d = b + 1$$
$$e = c * d$$

Compute: $\frac{\delta e}{\delta a}$ & $\frac{\delta e}{\delta b}$

Demo 1: Using Tensors & Autograd

Computational Graph



Demo 2: One Layer Network

Write Python code to program a one-layered network, that contains a single linear layer, with ‘in_dim’ inputs and ‘out_dim’ outputs. Give the network a random input, print the output. Also print the weights of the network.

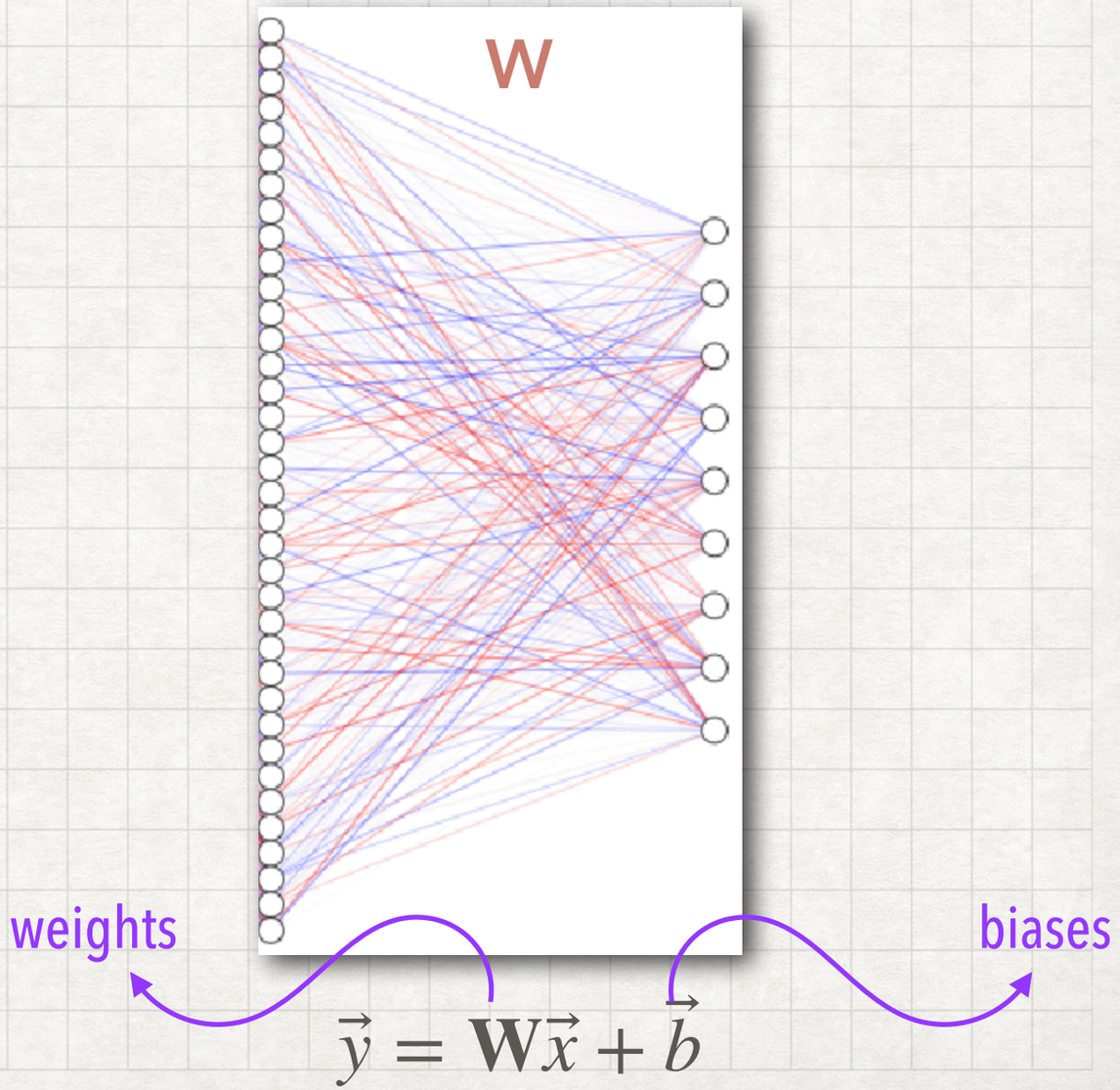
Pseudocode:

Define a class MyOneLayerdNet that:

- Create a Linear layer (matrix multiply + bias)
- Store it in self.fc

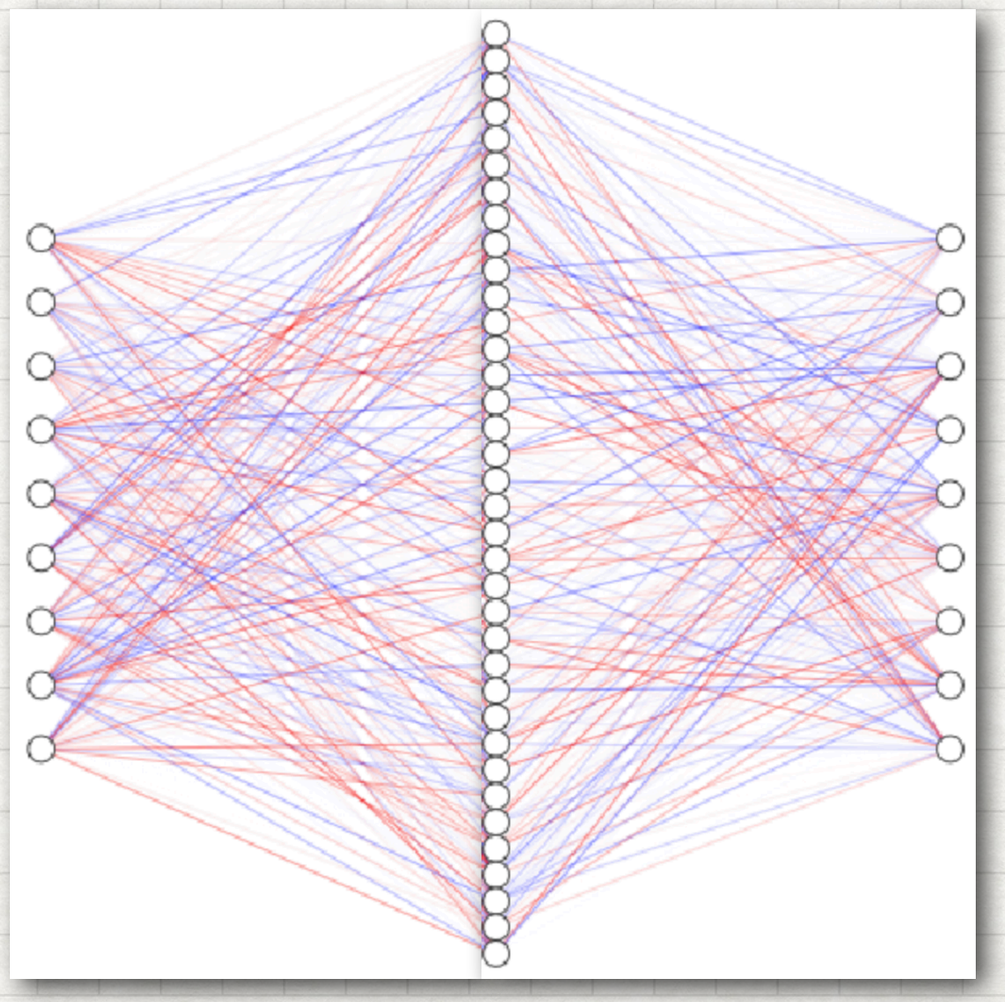
In forward(x):

- Apply the Linear layer to x
- Return the output



Demo 3: Small MLP

Write Python code to program a two-layered network, that contains a linear layer, followed by a ReLU function, which is then followed by another Linear layer. The output will be a set of "logits" (pre-softmax scores). Give the network a random input and see what the outputs are.



Demo 4: Learning

Write Python code to simulate one training step. This involves choosing a loss function, choosing an optimisation method (e.g. gradient descent), forward pass and change parameters based on gradients.

Pseudocode:

1. CREATE a small batch of training data
   - x : batch of input vectors; y : correct class labels

2. DEFINE a simple neural network
   - input → hidden; activation (ReLU); hidden → class scores

4. CHOOSE a loss function
   - CrossEntropyLoss (handles softmax + log loss)

5. CHOOSE an optimizer
   - SGD / Adam optimizer
   - connect it to the model's trainable parameters

6. ONE TRAINING STEP:
   a. CLEAR old gradients

   b. FORWARD PASS
      - feed input x through the model
      - get logits (raw class scores)

   c. COMPUTE LOSS
      - compare logits with true labels y

   d. BACKWARD PASS
      - autograd computes gradients of loss w.r.t. all model parameters

   e. UPDATE PARAMETERS
      - optimizer uses gradients to adjust weights